FEAT-ZD-48600 · REQ-07 Technical Requirement Document · Approved TRD

Requirement agent · item feat.trd · source hobasa-requirement-agent/outputs/FEAT-ZD-48600/feat.trd.md · approved by Architect sim.architect

feat.trd: FEAT-ZD-48600

Technical Requirement Document for FEAT-ZD-48600 (ZD-48600, "Show which overdue invoices belong to vendors that usually pay late"). This is a draft for the REQ-07 gate (Architect).

The TRD is drafted from three sources: the approved PRD (feat.prd), the approved prototype (feat.prototype) and the current architecture. The current architecture means org.architecture plus the code in the four app repositories, which this step read. Code is cited as `file:<GitHub repository>/<path>`, using these GitHub repository names: `ap-pipeline-sample` (package ap_pipeline), `ap-forecast-sample` (ap_forecast), `ap-aging-sample` (ap_aging) and `ap-portal-sample` (ap_portal). The TRD adds no scope beyond feat.prd S-1 to S-7. Statements marked **TRD design (agent judgement)** are this agent's technical proposal for the Architect to accept or change; no input states them as fact. TRD assumptions (T-A) and questions (TQ) are new in this step.

Field	Value	Source
Trace ID	FEAT-ZD-48600	record:brief; feat.prd
Trigger	Prototype Approved	record:brief
Owner / gate of this step	PM / Architect	record:brief
Inputs	feat.prd (approved PRD), feat.prototype (approved prototype), org.architecture (feature override)	record:brief
Code files read (ap_pipeline)	`src/ap_pipeline/ingest.py`, `transform.py`, `dq.py`, `pipeline.py`, `__init__.py`; `ci.py`	file:ap-pipeline-sample/… (cited per row below)
Code files read (ap_forecast)	`src/ap_forecast/predict.py`, `explain.py`, `models.json`, `__init__.py`; `ci.py`	file:ap-forecast-sample/… (cited per row below)
Code files read (ap_aging)	`src/ap_aging/aging.py`, `__init__.py`; `ci.py`	file:ap-aging-sample/… (cited per row below)
Code files read (ap_portal)	`src/ap_portal/index.js`; `package.json`; `ci.mjs`; `tests/portal.test.js`	file:ap-portal-sample/… (cited per row below)
Other files read	`requirements/ZD-48600.md` (org overrides); `org/org.templates.md`; `org/org.decision_log.md`; `memory/MEMORY.md`	file:hobasa-agentic-sdlc/…; record:memory
Repositories	ap_pipeline, ap_forecast, ap_aging, ap_portal	feat.prd; org.architecture
Coding lanes (routing)	ap_pipeline → Data Engineering; ap_forecast → Data Science; ap_aging → Backend; ap_portal → Frontend; integration and end-to-end across all four → Testing Agent	file:hobasa-agentic-sdlc/requirements/ZD-48600.md (org.routing_map)
Risk tier (advisory)	T2; "The risk label is advisory only."	feat.prd (header, NFR-7); file:hobasa-agentic-sdlc/requirements/ZD-48600.md (org.risk_map)
Architecture rule	"Keep every existing signature; add new functions."	org.architecture
Template	TRD template: components affected, interfaces/contracts, data, security and tenancy, testing approach, risks	file:hobasa-agentic-sdlc/org/org.templates.md

0. Current architecture baseline (findings from the code)

org.architecture names the functions in each repository. The code adds the facts below. They are stated as findings because the cited file shows them. Rows marked **Conflict** need an Architect decision (section 7).

#	Finding	Effect on this change	Source		
F-1	`ingest.parse_feed(csv_text)` returns each CSV row as a `dict` of strings (`csv.DictReader`). A column missing from the header is absent from the dict. A row with fewer fields than the header gets `None` for the missing fields (agent judgement on `csv.DictReader` behaviour).	A feed without a `paid_date` column (feed v2) yields rows with no `paid_date` key; the new code must treat a missing key, `None` and an empty value the same.	file:ap-pipeline-sample/src/ap_pipeline/ingest.py		
F-2	`transform.to_model(rows)` returns `(records, rejected)`. Each record is a plain `dict` with keys `id` (from column `invoice_id`), `vendor`, `due_date` (`date.fromisoformat`), `amount` (`Decimal(r["amount"])`, not quantised) and `paid` (`True` only when the `paid` column is `"true"`, case-insensitive). There is no `Invoice` class. ap_pipeline `__version__ = "0.4.0"`.	"`Invoice.paid_date`" in feat.prd (AC-1, AC-2) is implemented as a new `paid_date` key in this dict. Because `amount` is not quantised, a feed value such as `"980.5"` gives a one-decimal `Decimal`; the overdue list quantises it (IF-5.5).	file:ap-pipeline-sample/src/ap_pipeline/transform.py; file:ap-pipeline-sample/src/ap_pipeline/__init__.py; feat.prd (AC-1, AC-2, AC-18)		
F-3	A rejected row is `{"row": r, "reason": type(e).__name__}`, so today's reasons are exception type names such as `ValueError` or `KeyError`.	The required reason `'paid_date'` (AC-3) is not an exception type name; it needs its own code path.	file:ap-pipeline-sample/src/ap_pipeline/transform.py; feat.prd (AC-3)		
F-4	`dq.quality_report(records, rejected)` returns `rows`, `loaded`, `rejected` and `reject_reasons` (the sorted set of rejected `reason` values).	`'paid_date'` appears in `reject_reasons` with no change to `dq.py`.	file:ap-pipeline-sample/src/ap_pipeline/dq.py		

#	Finding	Effect on this change	Source		
F-5	`pipeline.run(csv_text)` returns a tuple `(records, quality_report(records, rejected))`, not only the records.	The end-to-end wiring takes `records` from the first element of the tuple.	file:ap-pipeline-sample/src/ap_pipeline/pipeline.py		
F-6	`predict_delay_days(history)` returns the rounded mean of past delays as an `int`, returns 0 when there is no history, and counts negative delays (early payments) as 0.	`vendor_risk([])` returns `'low'` (PQ-1: answers the direct-call case). Negative `delay_days` kept by AC-6 still reach `vendor_risk`, but they count as 0 in the prediction (R-5).	file:ap-forecast-sample/src/ap_forecast/predict.py; feat.prd (AC-6, PQ-1)		
F-7	`build_explanation_request(vendor, predicted_days, config=None)` builds the prompt "Explain in one sentence why vendor {vendor} is predicted to be paid {predicted_days} days late." and reads provider, model and `max_tokens` from `models.json` (provider `acme-llm`, model `acme-text-2`).	`risk` is added as a new last keyword argument. The prompt without `risk` stays byte-for-byte the same (AC-12).	file:ap-forecast-sample/src/ap_forecast/explain.py; file:ap-forecast-sample/src/ap_forecast/models.json		
F-8	ap_forecast exports `__all__ = ["predict_delay_days", "build_explanation_request"]`, `__version__ = "0.9.0"`.	`vendor_risk` must be added to `__all__`.	file:ap-forecast-sample/src/ap_forecast/__init__.py		
F-9	`days_past_due(due_date, as_of)` counts calendar days after the due date and returns 0 when not yet due.	Reused for `days` in the overdue list (D-2026-031, P-A1).	file:ap-aging-sample/src/ap_aging/aging.py; record:memory (D-2026-031)		
F-10	**Conflict.** `bucket_for(days)` returns `"1-30"` only when `days < 30`, so exactly 30 days goes to `"31-60"`.	`overdue_with_risk` must **not** use `bucket_for`. If it did, an invoice exactly 30 days past due would land with the "31-60" invoices, against AC-14. The existing bucket boundary is itself inconsistent with its "1-30" label (TQ-1, R-1).	file:ap-aging-sample/src/ap_aging/aging.py; feat.prd (AC-14); record:memory (L-007)		

#	Finding	Effect on this change	Source		
F-11	`aging_buckets` skips invoices whose `paid` is truthy and documents each invoice as `{'id', 'due_date': date, 'amount': Decimal, 'paid': bool}`, with no `vendor` key. ap_aging exports only `aging_buckets` in `__all__`; `__version__ = "1.4.0"`.	`overdue_with_risk` needs a `vendor` key on each invoice. ap_pipeline records carry `vendor` (F-2), so pipeline records can be passed in directly. `overdue_with_risk` must be added to `__all__`.	file:ap-aging-sample/src/ap_aging/aging.py; file:ap-aging-sample/src/ap_aging/__init__.py		
F-12	**Conflict.** ap_portal exports `formatAmount(value)`, which returns `Number(value \	\	0).toFixed(2)`. That formats through a binary floating-point number.	This conflicts with D-2026-044 and NFR-3. `renderOverdueRisk` must **not** use `formatAmount`. The existing helper is kept unchanged under the signature rule; whether to fix it is a separate decision (TQ-2, R-2).	file:ap-portal-sample/src/ap_portal/index.js; record:memory (D-2026-044, L-011); feat.prd (NFR-3)
F-13	`renderAgingSummary(buckets)` builds HTML by string templates and escapes nothing. Its labels are constants. ap_portal exports `{ VERSION, LABELS, formatAmount, renderAgingSummary }`; `VERSION = '2.3.0'` in both `index.js` and `package.json`.	`renderOverdueRisk` renders feed data (vendor names, ids), so the no-escaping pattern must not be copied as it is (OQ-B8, S-4 below).	file:ap-portal-sample/src/ap_portal/index.js; file:ap-portal-sample/package.json		

#	Finding	Effect on this change	Source		
F-14	Each repository's CI requires: compile/syntax check, unit tests passing, coverage of at least 90%, and static checks. Python repos flag `print`, `eval`, `exec`, bare `except` and lines over 140 characters. ap_portal flags `console.log`, `eval(` and lines over 140 characters. ap_portal tests use `node:test` and load the package as `require('ap_portal')` with `NODE_PATH=src`.	Every new function needs unit tests in its own repository to keep coverage at or above 90%.	file:ap-pipeline-sample/ci.py; file:ap-forecast-sample/ci.py; file:ap-aging-sample/ci.py; file:ap-portal-sample/ci.mjs; file:ap-portal-sample/tests/portal.test.js		
F-15	Runtimes: ap_pipeline, ap_forecast and ap_aging are Python 3.9; ap_portal is Node.js 22 CommonJS.	New Python code must not use syntax newer than 3.9, for example `match` or `X | Y` type unions (agent judgement on language versions).	org.architecture		

1. Components affected

ID	Repository / lane	Component	Change	Kind	PRD trace	Source
C-1	ap_pipeline / Data Engineering	`transform.to_model` (`src/ap_pipeline/transform.py`)	Reads the optional `paid_date` column and adds `paid_date` to each record. Rejects an invalid `paid_date` with reason `'paid_date'`. Signature and return shape unchanged.	Additive change to existing behaviour (CR-1)	AC-1, AC-2, AC-3	feat.prd (AC-1 to AC-3, NFR-4); F-2, F-3
C-2	ap_pipeline / Data Engineering	New `delays_by_vendor(records)`. TRD design (agent judgement): new module `src/ap_pipeline/delays.py`	Builds `{vendor: [delay_days, ...]}`	New function	AC-4, AC-5, AC-6	feat.prd (AC-4 to AC-6)
C-3	ap_pipeline	`ingest.parse_feed`, `dq.quality_report`, `pipeline.run`	No change. `'paid_date'` flows into `reject_reasons` by itself	None	AC-3	F-1, F-4, F-5

ID	Repository / lane	Component	Change	Kind	PRD trace	Source
C-4	ap_forecast / Data Science	New `vendor_risk(history)`. TRD design (agent judgement): new module `src/ap_forecast/risk.py`; added to `__all__`	Rule on `predict_delay_days(history)`; no LLM	New function	AC-7 to AC-10	feat.prd (AC-7 to AC-10); F-6, F-8
C-5	ap_forecast / Data Science	`build_explanation_request` (`src/ap_forecast/explain.py`)	Gains optional `risk=None` as the last parameter	Additive change to existing behaviour (CR-4)	AC-11, AC-12	feat.prd (AC-11, AC-12, NFR-4); F-7
C-6	ap_aging / Backend	New `overdue_with_risk(invoices, as_of, risk_by_vendor)` in `src/ap_aging/aging.py` (agent judgement: next to `days_past_due`, which it reuses); added to `__all__`	Overdue list with risk	New function	AC-13 to AC-18	feat.prd (AC-13 to AC-18); F-9, F-10, F-11
C-7	ap_aging / Backend	New `overdue_rows_to_json(rows)` (TRD design, agent judgement; see IF-6 and TQ-3)	Serialises the overdue list to the CT-1 JSON contract	New function	AC-18	feat.prd (AC-18, NFR-2, PQ-3)
C-8	ap_portal / Frontend	New `renderOverdueRisk(rows)` in `src/ap_portal/index.js`, exported	Renders the overdue risk table	New function	AC-19, AC-20, AC-21	feat.prd (AC-19 to AC-21); feat.prototype (DL-1 to DL-4, ST-1 to ST-5)
C-9	ap_portal	`formatAmount`, `renderAgingSummary`	No change in this item (see TQ-2)	None	NFR-4	F-12, F-13
C-10	Cross-repository / Testing Agent	End-to-end test harness	Feed → delays → risk → overdue list → JSON → portal table	New test	AC-22, AC-23	feat.prd (AC-22, AC-23); file:hobasa-agentic-sdlc/requirements/ZD-48600.md (org.routing_map)

Not affected: finance posting, authentication, tenancy, payment execution, LLM provider or model, aging export, the regulatory "Late" rule (feat.prd section 1 "Not in this change").

2. Interfaces and contracts

All Python signatures below use Python 3.9 typing (`typing.Dict`, `List`, `Optional`). "Days" are calendar days (D-2026-031).

IF-1 `transform.to_model(rows)` *(ap_pipeline, changed additively)*

Signature unchanged: `to_model(rows: List[Dict[str, str]]) -> Tuple[List[Dict], List[Dict]]` (F-2).

#	Rule	Trace	Source
IF-1.1	Read `raw = (r.get("paid_date") or "").strip()`. A missing column (feed v2), `None` and an empty value are treated the same.	AC-1	feat.prd (AC-1); F-1
IF-1.2	If `raw` is empty: `paid_date = None`.	AC-1	feat.prd (AC-1)
IF-1.3	If `raw` is non-empty, parse it with `date.fromisoformat(raw)`, the same parser used today for `due_date` (TRD design, agent judgement). On Python 3.9 this accepts the `YYYY-MM-DD` form (agent judgement on Python behaviour).	AC-2, P-A3	feat.prd (AC-2, P-A3); F-2
IF-1.4	If parsing fails, append `{"row": r, "reason": "paid_date"}` to `rejected` and add no record. The reason is the literal string `'paid_date'`, not an exception type name.	AC-3	feat.prd (AC-3); F-3
IF-1.5	Order of checks (TRD design, agent judgement): the existing fields are validated first, exactly as today. `paid_date` is checked only for a row that passes them. A row that fails both keeps today's reason, so the behaviour of existing rejects does not change. This narrows AC-3, which states no precedence; it applies only to rows that are invalid for two reasons, and it is raised as TQ-10.	AC-3, NFR-4	feat.prd (AC-3, NFR-4); F-3
IF-1.6	Each accepted record gains the key `paid_date` (`date` or `None`). All existing keys and values are unchanged.	AC-1, AC-2	feat.prd (AC-1, AC-2); file:hobasa-agentic-sdlc/requirements/ZD-48600.md (org.data_model)
IF-1.7	An unpaid row with a valid `paid_date` (OQ-B6, open): interim TRD design (agent judgement), not a decision. The row is accepted and the date is kept on the record as parsed. `delays_by_vendor` ignores it because the row is unpaid (IF-2.2). If OQ-B6 is answered "reject", only IF-1 changes.	OQ-B6	feat.prd (AC-2, OQ-B6)

IF-2 `delays_by_vendor(records)` *(ap_pipeline, new)*

```
delays_by_vendor(records: List[Dict]) -> Dict[str, List[int]]
```

#	Rule	Trace	Source
IF-2.1	Input: the `records` from `to_model` / `pipeline.run` (dicts with `vendor`, `due_date`, `paid`, `paid_date`).	AC-4, AC-23	feat.prd (AC-4); F-2, F-5

#	Rule	Trace	Source
IF-2.2	Include a record only when `record["paid"]` is true **and** `record.get("paid_date")` is not `None`.	AC-4, AC-5	feat.prd (AC-4, AC-5)
IF-2.3	`delay_days = (paid_date - due_date).days` (an `int`). A negative value is kept, with no floor.	AC-4, AC-6	feat.prd (AC-4, AC-6)
IF-2.4	Values are appended in record (feed) order. Keys are inserted in the order each vendor is first seen; Python 3.9 dicts keep insertion order.	AC-4	feat.prd (AC-4)
IF-2.5	A vendor with no qualifying record gets no key. With B-A3, its invoices then get `'unknown'` in the overdue list (AC-17).	AC-17, PQ-1	feat.prd (AC-17, B-A3, PQ-1)
IF-2.6	Pure function: no I/O, and it does not change `records`.	—	TRD design (agent judgement)

IF-3 `vendor_risk(history)` *(ap_forecast, new)*

```
vendor_risk(history: Iterable[int]) -> str
```

#	Rule	Trace	Source
IF-3.1	`p = predict_delay_days(history)`; return `'high'` if `p > 30`, else `'medium'` if `p > 10`, else `'low'`.	AC-7, AC-8, AC-9	feat.prd (AC-7 to AC-9); file:hobasa-agentic-sdlc/requirements/ZD-48600.md
IF-3.2	No LLM call, no network and no read of `models.json`.	AC-10	feat.prd (AC-10); file:hobasa-agentic-sdlc/requirements/ZD-48600.md (org.model_dependency_map)
IF-3.3	Returns only the three values above. `vendor_risk([])` returns `'low'` because `predict_delay_days([])` returns 0.	PQ-1	F-6; feat.prd (PQ-1)
IF-3.4	`predict_delay_days` returns a rounded `int`, so the comparisons run on integers. Python's `round` sends a mean exactly half-way between two integers to the even one (agent judgement on Python behaviour). This is noted for the boundary tests (T-6).	AC-8, AC-9	F-6

IF-4 `build_explanation_request(vendor, predicted_days, config=None, risk=None)` *(ap_forecast, changed additively)*

#	Rule	Trace	Source
IF-4.1	`risk` is added as the **last** parameter with default `None`, so existing positional and keyword calls are unchanged.	AC-12, NFR-4	feat.prd (AC-12, NFR-4); F-7; org.architecture

#	Rule	Trace	Source
IF-4.2	When `risk is not None`, the prompt is today's prompt followed by `f" Risk level: {risk}."`. It therefore ends with `' Risk level: <risk>.'`.	AC-11	feat.prd (AC-11)
IF-4.3	When `risk` is `None`, the prompt and the returned dict are identical to today's.	AC-12	feat.prd (AC-12); F-7
IF-4.4	Provider, model and `max_tokens` still come from `config` or `models.json`, unchanged.	OS (no provider change)	feat.prd (section 1); F-7
IF-4.5	"Given" means `risk is not None` (TRD design, agent judgement; TQ-4).	AC-11	feat.prd (AC-11)

IF-5 `overdue_with_risk(invoices, as_of, risk_by_vendor)` *(ap_aging, new)*

`overdue_with_risk(invoices: Iterable[Dict], as_of: date, risk_by_vendor: Dict[str, str]) -> List[Dict]`

#	Rule	Trace	Source
IF-5.1	Skip an invoice when `inv.get("paid")` is truthy, whatever its days past due. This is the same test `aging_buckets` uses.	AC-15	feat.prd (AC-15); F-11; record:memory (D-2026-050)
IF-5.2	`days = days_past_due(inv["due_date"], as_of)` (calendar days; existing function).	AC-13, P-A1	feat.prd (AC-13, P-A1); F-9
IF-5.3	Include the invoice only when `days > 30`. Exactly 30 is excluded. **Do not use** `bucket_for` (F-10).	AC-13, AC-14	feat.prd (AC-13, AC-14); F-10
IF-5.4	Row = `{"id": inv["id"], "vendor": inv["vendor"], "days": days, "amount": <IF-5.5>, "risk": risk_by_vendor.get(inv["vendor"], "unknown")}`. Exactly these five keys.	AC-16, AC-17	feat.prd (AC-16, AC-17)
IF-5.5	`amount` stays a `Decimal`. If the input is not already a `Decimal`, it is converted with `Decimal(...)`, as `aging_buckets` does, and never through `float`. It is then quantised to two decimal places with `quantize(Decimal("0.01"))`, because pipeline amounts are not quantised (F-2) and AC-18 requires a Decimal with 2 dp (TRD design, agent judgement on where to quantise).	AC-18, NFR-1	feat.prd (AC-18, NFR-1); F-2, F-11; record:memory (D-2026-044, L-011)
IF-5.6	Sort with key `(-row["days"], row["id"])`: days descending, then `id` ascending as text.	AC-16, P-A2	feat.prd (AC-16, P-A2)

#	Rule	Trace	Source
IF-5.7	Each invoice needs a `vendor` key. Pipeline records have one (F-2). A missing `vendor` raises `KeyError` (TRD design, agent judgement; TQ-9).	AC-16	F-2, F-11
IF-5.8	Pure function; `aging_buckets`, `days_past_due` and `bucket_for` are unchanged.	NFR-4	feat.prd (NFR-4); org.architecture

IF-6 CT-1: overdue list JSON contract (ap_aging → ap_portal)

feat.prd PQ-3 proposes a named contract for the TRD. TRD design (agent judgement), for Architect approval:

Field	JSON type	Rule	Source
`id`	string	Invoice id as in the record	feat.prd (AC-16); file:hobasa-agentic-sdlc/requirements/ZD-48600.md (org.data_model: "id (str)")
`vendor`	string	Vendor name as in the record	feat.prd (AC-16)
`days`	integer	Calendar days past due; always greater than 30	feat.prd (AC-13, AC-16, PQ-3: type not given, so this is agent judgement)
`amount`	string	Decimal quantised to two decimals, as plain text with no exponent and no thousands separator (for example `"1250.00"`)	feat.prd (AC-18, NFR-2); org.architecture; feat.prototype (section 6 dataset)
`risk`	string	One of `high`, `medium`, `low`, `unknown`	feat.prd (AC-17, AC-20)

- Payload: a JSON array of row objects, in the order of IF-5.6. An empty list is `[]`.
- CT-1 has no date field, so the "dates as ISO strings" rule (org.architecture) has nothing to apply to here. Any date added later must be an ISO string.
- Serialiser (C-7, TRD design): `overdue_rows_to_json(rows) -> str` in ap_aging. It turns `amount` into a string with `str(amount.quantize(Decimal("0.01")))` and keeps everything in `Decimal`. The code read shows no existing service or transport layer between ap_aging and ap_portal (`aging_buckets` returns Python `Decimal` values, and `renderAgingSummary` takes a plain object), so who owns serialisation and transport is open (TQ-3).
- Example (synthetic, from feat.prototype): `[{"id": "SYN-INV-004", "vendor": "Synthetic Vendor A", "days": 75, "amount": "1250.00", "risk": "high"}]`.

IF-7 `renderOverdueRisk(rows)` *(ap_portal, new)*

`renderOverdueRisk(rows) -> string`, exported from `src/ap_portal/index.js` next to the existing exports (F-13).

#	Rule	Trace	Source
IF-7.1	If `rows` is missing or empty, return `''`. No table, header or message.	AC-21	feat.prd (AC-21); feat.prototype (ST-2, DL-4)

#	Rule	Trace	Source
IF-7.2	Otherwise return `<table class="overdue-risk">`, then a header row `<tr><th>Invoice</th><th>Vendor</th><th>Days overdue</th><th>Amount</th></tr>` with no row class, then one `<tr class="risk-RISK">` per row in the given order, then `</table>`. TRD design (agent judgement): no `<thead>/<tbody>`, to match the plain style of `renderAgingSummary` (TQ-7).	AC-19	feat.prd (AC-19); feat.prototype (DL-1, DL-3, ST-1); F-13
IF-7.3	Each body row has four cells in the order id, vendor, days, amount. The amount cell uses `class="amount"`, as `renderAgingSummary` does (agent judgement).	AC-19	feat.prd (AC-19); F-13
IF-7.4	Row class: `risk-` + `row.risk`. TRD design (agent judgement): any value outside `high`, `medium`, `low`, `unknown` is rendered as `risk-unknown`, so feed data can never inject a class name (TQ-6).	AC-20	feat.prd (AC-20); feat.prototype (DL-2)
IF-7.5	Amount: shown from the CT-1 string **without** `Number()`, `parseFloat` or `formatAmount` (F-12). TRD design (agent judgement) for OQ-B10: a string-only helper checks that the value is a decimal string. If it has fewer than two decimal places, the helper pads it with zeros (for example `"980.5"` → `"980.50"`). A value that already has two decimals is shown unchanged. CT-1 always sends two decimals, so the padding is only a safeguard. It departs slightly from the prototype prompt ("Display the amount string exactly as given"), so it is raised as TQ-11.	AC-19, NFR-3, OQ-B10	feat.prd (NFR-3, OQ-B10); feat.prototype (section 1 "Amount display", section 7); record:memory (D-2026-044)
IF-7.6	`days` is rendered as its integer text.	AC-19	feat.prd (AC-19)
IF-7.7	TRD design (agent judgement), pending OQ-B8: HTML-escape `id` and `vendor` (`&`, `<`, `>`, `"`, `'`) because they come from feed data (F-13).	NFR-7, OQ-B8	feat.prd (NFR-7, OQ-B8)
IF-7.8	No visible Risk column or text and no row actions, following AC-19 while OQ-B9 is open.	AC-19, OQ-B9, NFR-6	feat.prd (AC-19, OQ-B9, NFR-6); feat.prototype (PQ-P4, PQ-P7)
IF-7.9	No `console.log` or `eval` (CI static check).	—	F-14

IF-8 End-to-end wiring (Testing Agent)

```
records, report = pipeline.run(csv_text)                              # F-5: run returns a tuple
```

```
delays          = delays_by_vendor(records)                          # IF-2
risk_by_vendor  = {v: vendor_risk(h) for v, h in delays.items()}     # IF-3, B-A3
rows            = overdue_with_risk(records, as_of, risk_by_vendor)  # IF-5
payload         = overdue_rows_to_json(rows)                         # IF-6 (CT-1)
html            = renderOverdueRisk(JSON.parse(payload))             # IF-7 (Node.js)
```

Sources: feat.prd (AC-22, AC-23, B-A3); F-2, F-5. Records from ap_pipeline already carry every key `overdue_with_risk` needs (`id`, `vendor`, `due_date`, `amount`, `paid`), so no mapping step is needed (F-2, F-11). The last step runs in Node.js 22. TRD design (agent judgement): the harness runs the Python stages, writes `payload`, and calls `node` to render it, so the cross-language contract (CT-1) is tested as real JSON.

3. Data

ID	Data item	Shape / type	Rule	Source
D-1	Vendor feed v3	CSV columns `invoice_id, vendor, due_date, amount, paid, paid_date`	`paid_date` optional, ISO date, empty when unpaid. Schedule unchanged: daily 02:00 Asia/Kolkata	file:hobasa-agentic-sdlc/requirements/ZD-48600.md (org.customer_mappings); feat.prd (NFR-9)
D-2	Invoice record (ap_pipeline output)	dict: `id` str, `vendor` str, `due_date` date, `amount` Decimal, `paid` bool, `paid_date` date or None	`paid_date` added by this change	file:hobasa-agentic-sdlc/requirements/ZD-48600.md (org.data_model); F-2
D-3	Rejected row	dict: `row` (raw dict), `reason` str	New reason value `'paid_date'`	F-3; feat.prd (AC-3)
D-4	Delay history	`Dict[str, List[int]]`	Paid records with a `paid_date` only; feed order; negatives kept	feat.prd (AC-4 to AC-6)
D-5	Risk map `risk_by_vendor`	`Dict[str, str]`	One entry per vendor in D-4; values `high` / `medium` / `low`	feat.prd (B-A3, AC-7 to AC-9)
D-6	Overdue row (in ap_aging)	dict: `id` str, `vendor` str, `days` int, `amount` Decimal (2 dp), `risk` str	Unpaid, `days > 30`, sorted days descending then id	feat.prd (AC-13 to AC-18)
D-7	Overdue list JSON (CT-1)	array of objects; `amount` two-decimal string	See IF-6	feat.prd (AC-18, NFR-2); org.architecture
D-8	Explanation request	dict `provider`, `model`, `max_tokens`, `messages`	Prompt may end with `'Risk level: <risk>.'`	F-7; feat.prd (AC-11)

Data rules:

- Money: `Decimal` end to end in Python, and a string in JSON and in ap_portal; never `float` or a JS `Number` (D-2026-044, L-011, NFR-1 to NFR-3; F-12).
- Days: calendar days after the due date (D-2026-031; F-9).
- Paid invoices: excluded from the overdue list (D-2026-050; IF-5.1).
- No storage, schema migration or new persistent data. Every new function is in-memory and pure (agent judgement; the code read has no database layer).
- Backward compatibility: a feed without `paid_date` (v2) loads as before, with `paid_date = None` (IF-1.1).

4. Security and tenancy

ID	Requirement	Source
S-1	No finance posting, authentication, tenancy or payment execution is added or changed; tier T2.	file:hobasa-agentic-sdlc/requirements/ZD-48600.md (org.risk_map); feat.prd (NFR-7)

ID	Requirement	Source
S-2	The risk label is advisory only. No code path acts on a risk value: no posting, payment or automatic chase. ap_portal shows no row actions.	feat.prd (NFR-6); feat.prototype (PQ-P7)
S-3	`vendor_risk` uses no LLM, so the risk level can be traced to a fixed rule (auditability).	feat.prd (AC-10, NFR-5); file:hobasa-agentic-sdlc/requirements/ZD-48600.md (org.model_dependency_map)
S-4	Output encoding: `renderOverdueRisk` escapes `id` and `vendor` and restricts the row class to the four known values (IF-7.4, IF-7.7). This is the TRD default pending OQ-B8 (agent judgement: feed text rendered as HTML is an injection risk).	feat.prd (NFR-7, OQ-B8); F-13
S-5	External data flow: with `risk`, the explanation prompt sent to the configured provider (`acme-llm`) also carries the vendor's risk label. The vendor name is already sent today. Agent judgement: the Architect should confirm that sending this label to the provider is acceptable (TQ-8).	F-7; file:ap-forecast-sample/src/ap_forecast/models.json
S-6	No secrets, credentials or personal data are added. Tests and examples use synthetic data only (`SYN-INV-*`, "Synthetic Vendor <letter>").	feat.prototype (SD-1, SD-6)

5. Testing approach

Each lane adds unit tests in its own repository. The repository CI must stay green: compile, tests, coverage of at least 90%, and static checks (F-14). Boundary cases are tested explicitly (L-007, NFR-8). Test values below are illustrative synthetic data (agent judgement), except those quoted from feat.prototype.

ID	Repo / lane	Test	Covers	Source
T-1	ap_pipeline	Feed with `paid_date` column: empty value gives `paid_date is None`; valid ISO date on a paid row is parsed; feed with no `paid_date` column (v2) loads with `None`	AC-1, AC-2, IF-1.1	feat.prd (AC-1, AC-2); F-1
T-2	ap_pipeline	Invalid `paid_date` (for example `"2026-13-01"`, `"not-a-date"`) rejects the row with `reason == 'paid_date'`, and `quality_report` lists `'paid_date'` in `reject_reasons`	AC-3, IF-1.4	feat.prd (AC-3); F-3, F-4
T-3	ap_pipeline	A row with an invalid `due_date` and an invalid `paid_date` keeps today's reason (IF-1.5, pending TQ-10); existing tests still pass unchanged	NFR-4	feat.prd (NFR-4)
T-4	ap_pipeline	`delays_by_vendor`: only paid records with a date; paid without a date excluded and not rejected; negative delay kept; feed order per vendor; unpaid row with a `paid_date` excluded	AC-4, AC-5, AC-6, IF-1.7	feat.prd (AC-4 to AC-6, OQ-B6)

ID	Repo / lane	Test	Covers	Source
T-5	ap_forecast	`vendor_risk` returns `'high'`, `'medium'`, `'low'`; makes no LLM or config call (check, for example, that `load_config` is never called)	AC-7 to AC-10	feat.prd (AC-7 to AC-10); F-7
T-6	ap_forecast	Boundaries: a history predicting exactly 30 gives `'medium'`, exactly 10 gives `'low'`, and the next integer above each gives the higher level; an empty history gives `'low'`; a history of early payments only (negative delays) gives `'low'`	AC-8, AC-9, NFR-8, F-6	feat.prd (AC-8, AC-9, NFR-8); record:memory (L-007); F-6
T-7	ap_forecast	`build_explanation_request` with `risk` ends with `' Risk level: <risk>.'`; without `risk` the result equals today's output exactly; positional call with `config` still works	AC-11, AC-12	feat.prd (AC-11, AC-12); F-7
T-8	ap_aging	Unpaid exactly 30 days past due is **not** listed; 31 days is listed; paid invoice far past due is not listed; not-yet-due is not listed	AC-13, AC-14, AC-15	feat.prd (AC-13 to AC-15); feat.prototype (section 6: SYN-INV-001 at 31, SYN-INV-005 at 30); record:memory (L-007, D-2026-050)
T-9	ap_aging	Row keys are exactly `id, vendor, days, amount, risk`; sort by days descending then id ascending (a tie on `days`, as in the prototype's SYN-INV-002 / SYN-INV-003); vendor missing from `risk_by_vendor` gives `'unknown'`	AC-16, AC-17	feat.prd (AC-16, AC-17); feat.prototype (section 6, ST-5)
T-10	ap_aging	`amount` is a `Decimal` quantised to two decimals (also when the input `Decimal` has one decimal); `overdue_rows_to_json` emits `"1250.00"` for the prototype's SYN-INV-004 amount and a two-decimal string for every row; an empty list gives `[]`	AC-18, NFR-1, NFR-2	feat.prd (AC-18, NFR-1, NFR-2); feat.prototype (section 6)
T-11	ap_aging	`aging_buckets`, `days_past_due` and `bucket_for` results are unchanged (regression)	NFR-4	feat.prd (NFR-4); F-10

ID	Repo / lane	Test	Covers	Source
T-12	ap_portal	Non-empty rows: output starts with `<table class="overdue-risk">`, has the four headings in order, and one `<tr class="risk-…">` per row in input order; each of the four risk values gives its class	AC-19, AC-20	feat.prd (AC-19, AC-20); feat.prototype (WF-1)
T-13	ap_portal	Empty array gives `''`	AC-21	feat.prd (AC-21); feat.prototype (WF-2)
T-14	ap_portal	Amount string shown unchanged (`"1250.00"`, `"15.00"`); a string with more significant digits than a JS `Number` can hold is shown unchanged, which proves no float conversion; `"980.5"` is padded to `"980.50"` (pending TQ-11)	NFR-3, IF-7.5	feat.prd (NFR-3); feat.prototype (section 6); F-12
T-15	ap_portal	A vendor name containing <, & or " is escaped; an unexpected `risk` value renders `risk-unknown`	IF-7.4, IF-7.7, OQ-B8	feat.prd (NFR-7, OQ-B8)
T-16	ap_portal	Existing `formatAmount` and `renderAgingSummary` tests still pass	NFR-4	file:ap-portal-sample/tests/portal.test.js
T-17	Cross-repo / Testing Agent	End to end on a synthetic feed (IF-8): the rendered table's rows (ids, vendors, days, amounts, classes, order) match `overdue_with_risk` built from the same feed; the feed includes an exactly-30-days invoice, a paid invoice, an invalid `paid_date` row and a vendor with no paid history	AC-22, AC-23, P-A4	feat.prd (AC-22, AC-23, P-A4); file:hobasa-agentic-sdlc/requirements/ZD-48600.md (org.routing_map)

Protected test oracles owned by QA and SMEs may add their own expectations. Where an IF rule above is agent judgement (for example IF-7.2 header form, IF-7.4, IF-7.7), the oracle wins and the rule changes.

6. Risks

ID	Risk	Impact	Mitigation	Source
R-1	`bucket_for(30)` returns `"31-60"` (F-10). The existing aging summary and the new overdue list will therefore treat an invoice exactly 30 days past due differently.	Users may see an invoice in the "31-60 days" bucket that is not in the overdue risk table.	IF-5.3 forbids reusing `bucket_for`; T-8 and T-11. Whether to fix `bucket_for` is a separate decision (TQ-1).	file:ap-aging-sample/src/ap_aging/aging.py; feat.prd (AC-14); record:memory (L-007)

ID	Risk	Impact	Mitigation	Source
R-2	The exported `formatAmount` formats through a binary float (F-12), against D-2026-044.	A developer may reuse it in `renderOverdueRisk`; the existing aging summary already rounds through a float.	IF-7.5 forbids it and T-14 catches it. The fix to the existing helper is a separate decision (TQ-2).	file:ap-portal-sample/src/ap_portal/index.js; record:memory (D-2026-044)
R-3	PROV-ACME-2026-11 also changes `build_explanation_request` (PQ-5).	Merge conflict, or a lost `risk` parameter.	Order the two changes; T-7 guards the behaviour.	feat.prd (PQ-5)
R-4	CR-1 needs feed v3, while ZD-48470 brings feed v2 (PQ-6).	`paid_date` may be absent in production feeds, giving empty delay histories and all rows `'unknown'`.	IF-1.1 keeps v2 feeds loading. Availability is confirmed before release.	feat.prd (PQ-6)
R-5	`predict_delay_days` counts negative delays as 0 (F-6), while AC-6 keeps negatives in the history.	Early payments do not lower a vendor's predicted delay below zero. This is consistent with both sources, but may surprise users.	Documented here; T-6 covers it; TQ-5.	file:ap-forecast-sample/src/ap_forecast/predict.py; feat.prd (AC-6)
R-6	Rounding in `predict_delay_days` near the 30 and 10 thresholds (IF-3.4).	Mean delays just above a threshold can be classed at the lower level.	Boundary tests T-6 on the rounded value.	file:ap-forecast-sample/src/ap_forecast/predict.py; feat.prd (AC-8, AC-9)
R-7	No transport layer between ap_aging and ap_portal exists in the code read (IF-6).	The CT-1 contract could be implemented twice or not at all.	C-7 puts the serialiser in ap_aging; TQ-3.	file:ap-aging-sample/src/ap_aging/aging.py; file:ap-portal-sample/src/ap_portal/index.js; feat.prd (PQ-3)
R-8	Feed text rendered as HTML without escaping (F-13 pattern).	Injection into the portal page.	IF-7.7 / S-4 / T-15, pending OQ-B8.	feat.prd (NFR-7, OQ-B8); file:ap-portal-sample/src/ap_portal/index.js
R-9	The end-to-end test crosses Python 3.9 and Node.js 22.	Harness setup is fragile.	IF-8 harness design; owned by the Testing Agent.	org.architecture; file:hobasa-agentic-sdlc/requirements/ZD-48600.md (org.routing_map)
R-10	No performance, volume or availability targets (PQ-4, NFR-11).	Sorting and rendering a large list is not sized.	Carried as an open question; no target is invented.	feat.prd (NFR-11, PQ-4)
R-11	OQ-B6 (unpaid row with a `paid_date`) is open.	IF-1.7 is an interim design.	A localised change in IF-1 if answered "reject".	feat.prd (OQ-B6)

7. Assumptions and open questions

7.1 TRD assumptions (agent judgement; confirm at the gate)

ID	Assumption	Used in
T-A1	CT-1 as in IF-6: `days` is a JSON integer; `amount` is a quantised two-decimal string.	IF-6, IF-7
T-A2	`risk` is "given" when it is not `None`; it is the last parameter.	IF-4
T-A3	Existing reject reasons take precedence over `'paid_date'` (TQ-10).	IF-1.5

ID	Assumption	Used in
T-A4	New modules `src/ap_pipeline/delays.py` and `src/ap_forecast/risk.py`; `overdue_with_risk` and `overdue_rows_to_json` in `src/ap_aging/aging.py`; `renderOverdueRisk` in `src/ap_portal/index.js`.	Section 1
T-A5	Header row as `<tr><th>…</th></tr>` with no class; amount padding and HTML escaping as in IF-7.5 and IF-7.7.	IF-7
T-A6	No input requires a version change. Agent judgement: each changed package (ap_pipeline `0.4.0`, ap_forecast `0.9.0`, ap_aging `1.4.0`, ap_portal `2.3.0`; see F-2, F-8, F-11, F-13) takes a minor version step, because only additive functions are added. This is for the coding lanes to confirm.	Packaging of C-1 to C-8

7.2 New in this step (non-blocking, for the Architect)

ID	Question	Why it matters
TQ-1	`bucket_for(30)` returns `"31-60"`, which does not match its own "1-30" label or the `> 30` overdue rule (F-10). Fix it as a separate item (it changes aging totals), or accept it?	Consistency between the aging summary and the overdue table
TQ-2	`formatAmount` uses `Number(...).toFixed(2)` against D-2026-044 (F-12). Raise a separate item to fix the existing helper?	Decision-log compliance of existing code
TQ-3	Approve CT-1 and say who owns serialisation and transport between ap_aging and ap_portal (C-7 proposes ap_aging)	Interface between two coding lanes (PQ-3)
TQ-4	Confirm T-A2 (`risk is not None` means "given")	Exact behaviour of AC-11
TQ-5	Is counting early payments as 0 in the risk prediction (F-6) acceptable for this feature?	Meaning of the risk level
TQ-6	Confirm the row-class allow-list (unexpected `risk` renders `risk-unknown`)	AC-20 robustness
TQ-7	Confirm the header-row form (IF-7.2) against the QA oracle	Test alignment
TQ-8	Is sending the vendor's risk label to the external LLM provider acceptable (S-5)?	Data flow to a third party
TQ-9	Should `overdue_with_risk` reject an invoice without `vendor` (KeyError) or treat it as `'unknown'`?	Robustness of IF-5
TQ-10	A row whose required fields are invalid **and** whose `paid_date` is invalid: should it keep today's reason (IF-1.5, T-A3) or get `'paid_date'` (a literal reading of AC-3)?	Exact reject reason; existing reject behaviour
TQ-11	Accept the string-only padding of amounts with fewer than two decimals in ap_portal (IF-7.5), or display the CT-1 string strictly as received (feat.prototype prompt)?	Answer to OQ-B10

7.3 Carried (unchanged)

From feat.prd: OQ-B1 to OQ-B10 and PQ-1 to PQ-6. This TRD gives a technical proposal for OQ-B8 (IF-7.7), OQ-B10 (IF-7.5), PQ-1 (IF-3.3, IF-2.5) and PQ-3 (IF-6), but leaves each open for confirmation. From feat.prototype: PQ-P1 to PQ-P7. Also carried, as listed in feat.prd: feat.fitment FQ-3, FQ-5 and FQ-6; OQ-1 to OQ-5 from feat.classification; and Q-1 to Q-5 from feat.prebacklog.

8. Traceability (PRD to TRD)

PRD item	TRD coverage
AC-1, AC-2, AC-3	C-1; IF-1; D-1, D-2, D-3; T-1, T-2, T-3
AC-4, AC-5, AC-6	C-2; IF-2; D-4; T-4; R-5
AC-7, AC-8, AC-9, AC-10	C-4; IF-3; D-5; S-3; T-5, T-6; R-6
AC-11, AC-12	C-5; IF-4; D-8; T-7; R-3
AC-13 to AC-18	C-6, C-7; IF-5, IF-6; D-6, D-7; T-8 to T-11; R-1, R-7
AC-19, AC-20, AC-21	C-8; IF-7; T-12 to T-16; R-2, R-8
AC-22, AC-23	C-10; IF-8; T-17; R-9
NFR-1, NFR-2, NFR-3	IF-5.5, IF-6, IF-7.5; data rules; T-10, T-14
NFR-4	IF-1.5, IF-4.1, IF-5.8; C-3, C-9; T-3, T-11, T-16
NFR-5, NFR-6, NFR-7	S-1 to S-4
NFR-8	T-6, T-8
NFR-9	D-1
NFR-10	Only "overdue" / "Days overdue" are used (IF-7.2)
NFR-11	R-10
Prototype DL-1 to DL-4, ST-1 to ST-5	IF-7.1 to IF-7.8; T-12, T-13
Prototype SD-1 to SD-6	S-6; test data in section 5

9. Validation check

Check	Result	Source
TRD template sections present: components affected, interfaces/contracts, data, security and tenancy, testing approach, risks	Pass. Each template section has its own section in this document.	file:hobasa-agentic-sdlc/org/org.templates.md
Every PRD acceptance criterion and NFR is covered	Pass (section 8).	feat.prd
Prototype design rules and states covered	Pass (section 8).	feat.prototype
"Keep every existing signature; add new functions"	Pass. Only CR-1 (`to_model` behaviour) and CR-4 (`risk=None`) touch existing functions, and both are additive; signatures are unchanged.	org.architecture; feat.prd (NFR-4)
Current code read and cited, not assumed	Pass. Findings F-1 to F-15 cite the files listed in the header table.	file:ap-pipeline-sample/…, file:ap-forecast-sample/…, file:ap-aging-sample/…, file:ap-portal-sample/…
No new scope	Pass. Three new helpers are not named in feat.prd, and each only implements a PRD criterion. The CT-1 serialiser (C-7) implements AC-18 (TQ-3). The string-only amount helper (IF-7.5) implements NFR-3 (TQ-11). The HTML-escape helper (IF-7.7) implements NFR-7, pending OQ-B8. No user-visible behaviour is added.	feat.prd (AC-18, NFR-3, NFR-7, PQ-3, OQ-B8)
Decision log: D-2026-031 (calendar days)	Consistent (IF-5.2, F-9).	file:hobasa-agentic-sdlc/org/org.decision_log.md; record:memory

Check	Result	Source
Decision log: D-2026-044 (Decimal, never float)	Consistent for new code (IF-5.5, IF-6, IF-7.5). **Existing conflict found** in `formatAmount` (F-12), raised as TQ-2, not silently changed.	file:hobasa-agentic-sdlc/org/org.decision_log.md; file:ap-portal-sample/src/ap_portal/index.js
Decision log: D-2026-050 (paid excluded)	Consistent (IF-5.1).	file:hobasa-agentic-sdlc/org/org.decision_log.md
Lessons L-007, L-011	Applied (T-6, T-8; IF-5.5, IF-7.5).	record:memory
Conflict with an approved output	None found. The `bucket_for` boundary (F-10) is a conflict inside existing code, not with feat.prd; it is raised as TQ-1.	feat.prd; feat.prototype
Regulation citation	Not applicable: the BRD records that no regulatory source applies (carried in feat.prd).	feat.prd (section 8)